CS 499 Milestone Three Narrative
Algorithms and Data Structures Enhancement

Aaron Vogel
September 27, 2026

The artifact I enhanced is LumberTrack Inventory, an Android inventory application I created in CS 360. It is written in Java and uses SQLite to store user accounts and inventory records. The app lets a user create an account and add, edit, view, and delete lumber inventory items. It also supports optional SMS alerts when an item reaches its low-stock amount.

The original application worked, but it pulled inventory from the database and handled records mainly one row at a time. It did not keep a clear in-memory structure for quickly finding an item or identifying which low-stock item needed attention first.

On a job, running short on something as basic as screws or plywood can hold up the work. I have worked in construction, so keeping track of lumber, hardware, and supplies is familiar to me. The update lets the app find the item I am editing by its ID. It also puts the material furthest below its low-stock amount at the top.

I added an InventoryDataManager class that keeps three structures synchronized. An ArrayList stores inventory in alphabetical order for the table on the screen. A HashMap uses the SQLite item ID as the key so the app can quickly find an item when the user selects Edit. A PriorityQueue keeps only low-stock items and places the item with the biggest shortage at the front. The inventory screen now displays the current highest-priority low-stock item.

The main thing I had to watch was keeping the inventory information lined up after a change. For example, if I update an item’s quantity, it should no longer show as the highest low-stock item if the quantity is now okay. I added basic tests to check that the app finds the correct item by ID, ranks low-stock items correctly after updates, and keeps the structures synchronized after an item is deleted. Both the original and enhanced code are included in the ZIP so the improvements are easy to compare.

One trade-off is the cost of updating the priority queue. If an item's quantity changes then the app will remove the old item and re-add it back in (if it's still low stock). A peek() of the highest priority item is O(1), adding an item is O(log k) and remove(Object) of a specific item is O(k) where k is the number of items in the queue. That means an update could be O(k) as the queue grows. I was happy to accept that cost in order to keep the queue accurate, though it may be more significant if the inventory becomes very large. My tutor's feedback was really useful in helping me see that trade-off.

This enhancement directly supports the course outcome about designing and evaluating computing solutions with algorithmic principles and appropriate data structures. I chose each structure based on what the app needs to do. Searching an inventory display list by ID can take O(n) time, while the HashMap provides average O(1) lookup. The PriorityQueue provides O(1) access to the highest-priority low-stock item, while the inventory list is still available for normal alphabetical display. Loading inventory requires a final O(n log n) sort, which is a reasonable tradeoff for a small business inventory application.

The artifact also supports the outcomes related to professional communication and implementing solutions that provide value. The code is separated into an item class, a data manager, and the activity that controls the screen. The included analysis and unit tests make the design decisions easier to explain and verify.

LumberTrack uses a login, local SQLite data, and an optional low-stock text alert. I did not touch any of that for this milestone but i did changed how the app handles inventory instead. It can find the right item for Edit and put the lowest item at the top of the low-stock list. Travlr is where I did the security work: protected routes, cleaner registration email input, and tests for missing information and bad tokens.

When I clicked Edit on an item, I needed the app to find that exact record instead of searching through every row on the screen. The HashMap does that with the SQLite item ID. I kept the ArrayList because the screen still needs an alphabetical list of all the items. The low-stock part only needs to show the item furthest below its saved low-stock amount, so that is where I used the PriorityQueue. Those are three different jobs, so I stopped trying to make one structure handle all of them.

One problem I ran into was what happened after I edited an item. If I changed the quantity on a box of screws from 2 to 20, the old version could not stay in the low-stock queue. Otherwise, the app would still show it as the top priority even though it was no longer low. I fixed that by removing the old item and adding the updated item each time the quantity changes. When the inventory screen opens, it reads from SQLite again so the list, map, and queue all start with the same information.